Towards Java Virtual Machine for Processing-in-Memory

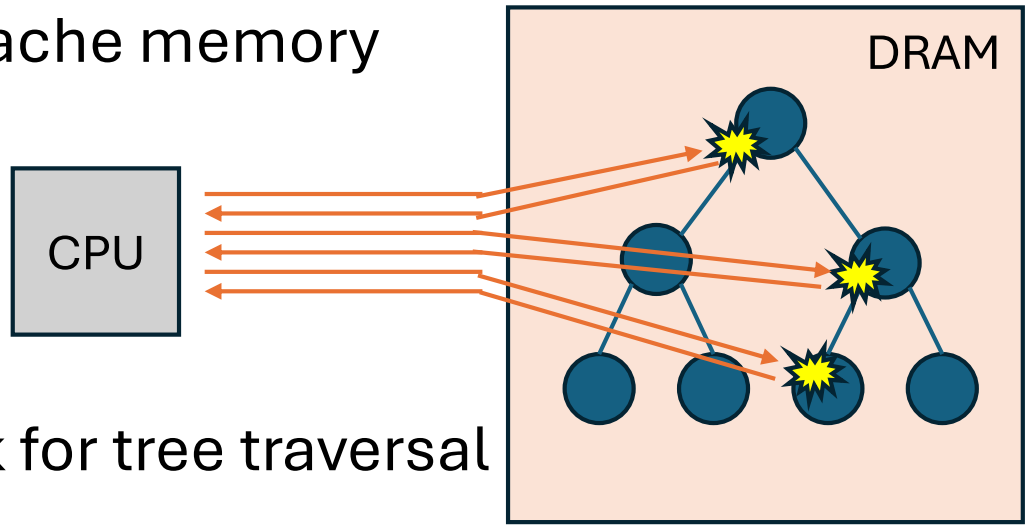
Kazuki Ichinose (Fixstars Corporation, Japan)

Shigeyuki Sato (University of Electro-Communications, Tokyo)

Tomoharu Ugawa (University of Tokyo)

Memory Wall

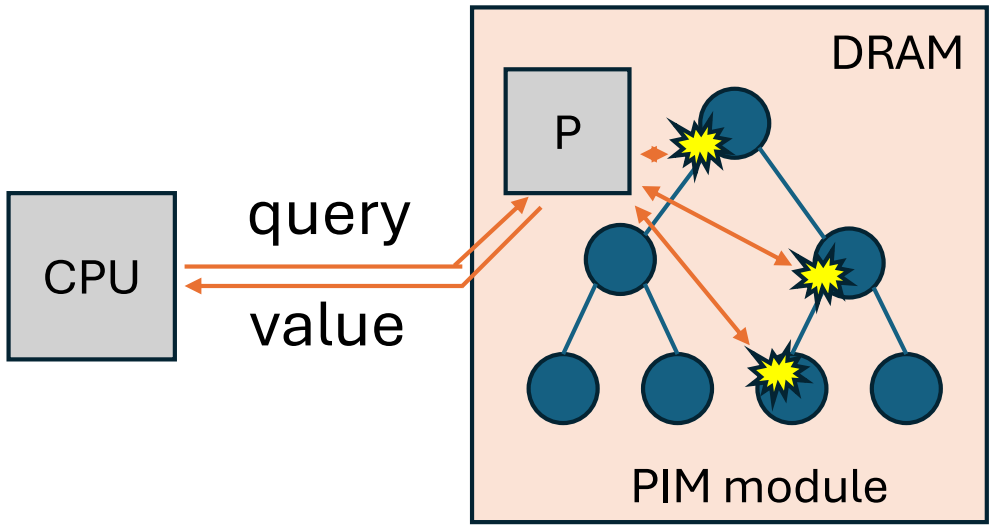
- Data-intensive applications are common
  - data analysis, LLM
- Data-transfer between CPU and DRAM is bottleneck of speed and energy efficiency
  - limitation on band width of memory bus
  - long physical distance to travel
  - multi-layer cache memory



Cache does not work for tree traversal

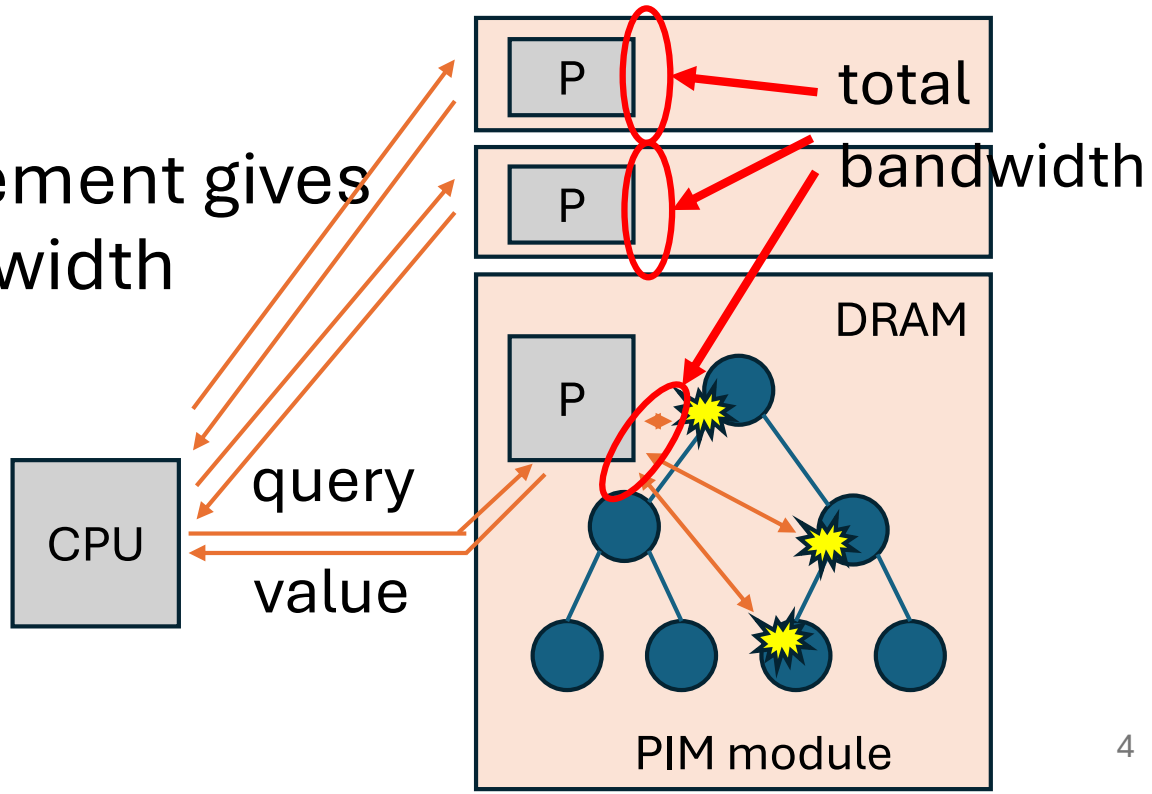
Processing-in-Memory (PIM)

- Approach to place processors in/near memory
- Offloading memory intensive workload to memory processors reduces data transfer



Processing-in-Memory (PIM)

- Approach to place processors in/near memory
- Offloading memory intensive workload to memory processors reduces data transfer
- Parallel arrangement gives high total bandwidth

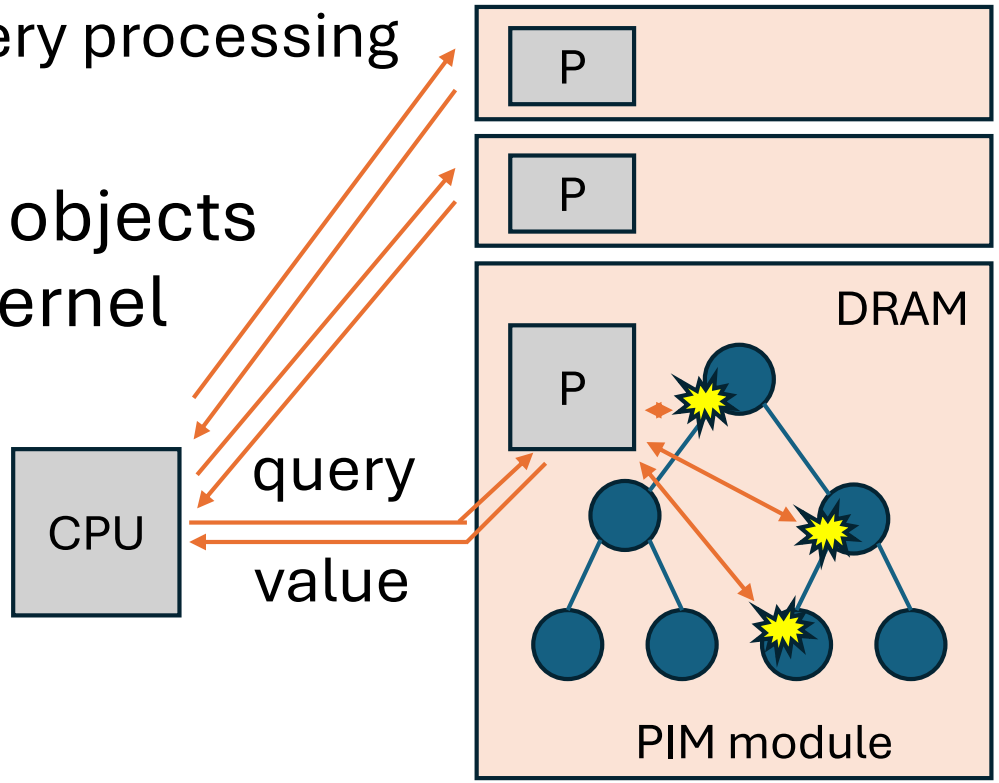


PIM Accelerators

- Two kinds of PIM accelerators
  - Domain specific PIM (i.e., for AI)
    - HBM-PIM (Samsung)
    - AiM (SK Hynix)
  - General purpose (GP) PIM
    - UPMEM PIM (UPMEM)
- Processors of GP-PIM supports
  - Full-set ISA
  - flexible parallelism (SPMD)

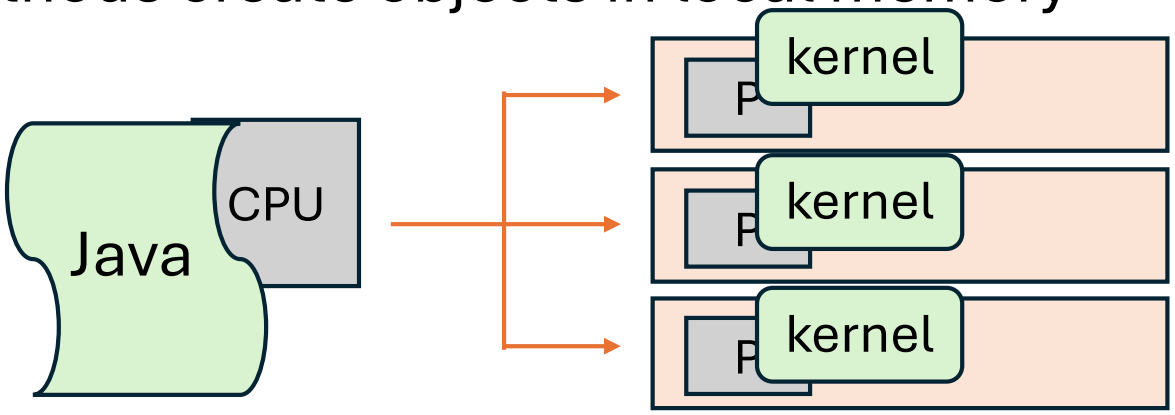
Can We Use GP-PIM in Java?

- Offload computation kernel of Java program to GP-PIM Accelerator
- Irregular computation as well as array programming
  - e.g. batched query processing on trees
- Want to use Java objects in computation kernel



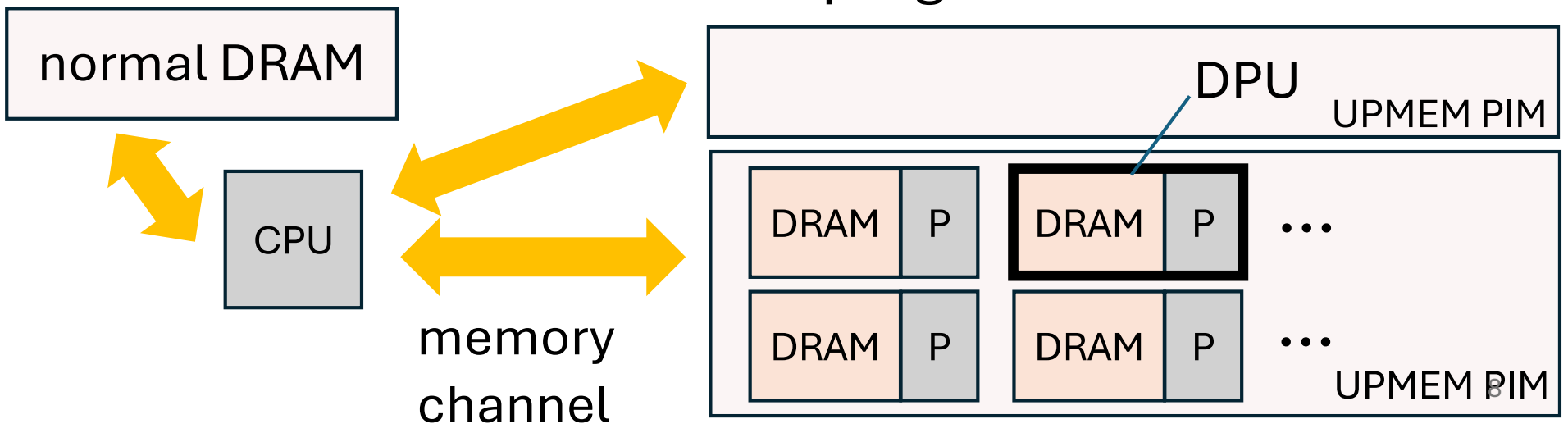
Yoshitsune (義経): Java Framework for UPMEM PIM

- Accelerates data-parallel computation
- Offload memory-intensive kernel method to UPMEM PIM
- Kernel methods run at all processors in parallel
- Avoid data transfer
  - Kernel methods use only objects in local memory
  - Kernel methods create objects in local memory



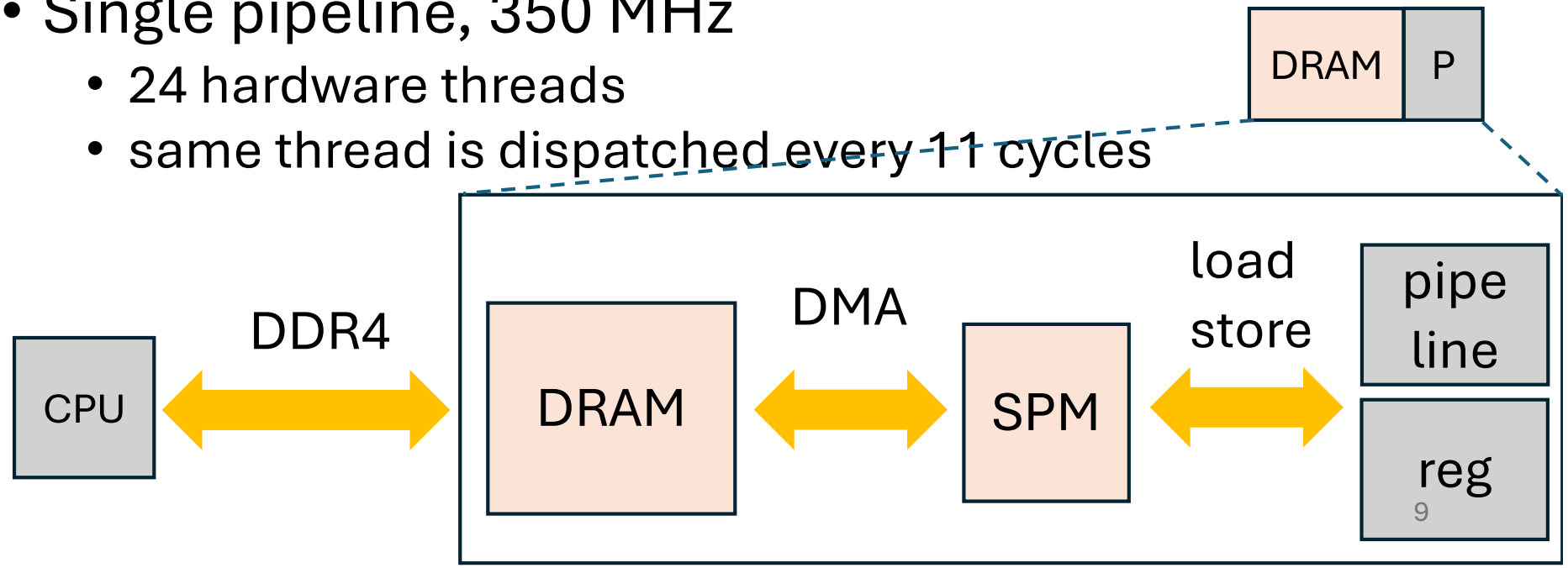
UPMEM PIM

- Works as a DRAM module, connected to the memory channel
  - CPU reads and writes though DDR4 interface
- Each memory bank has a processor, forming DPU
  - 2560 DPUs in a system with 20 UPMEM PIM module
- DPUs execute the same programs at once



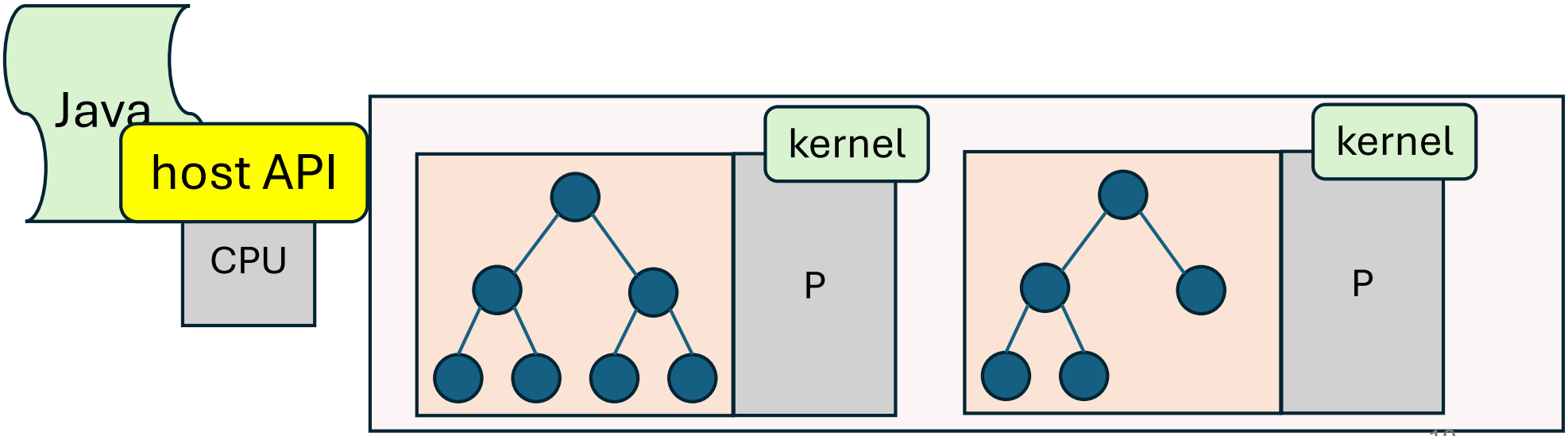
DPU

- 64 MB DRAM accessed using the single DMA unit
  - ~ 70 + 0.5 *n* cycles to transfer *n* bytes
  - contents are preserved after execution
- 64 KB scratch pad memory (SPM)
- No memory shared with other DPUs
- Single pipeline, 350 MHz
  - 24 hardware threads
  - same thread is dispatched every 11 cycles



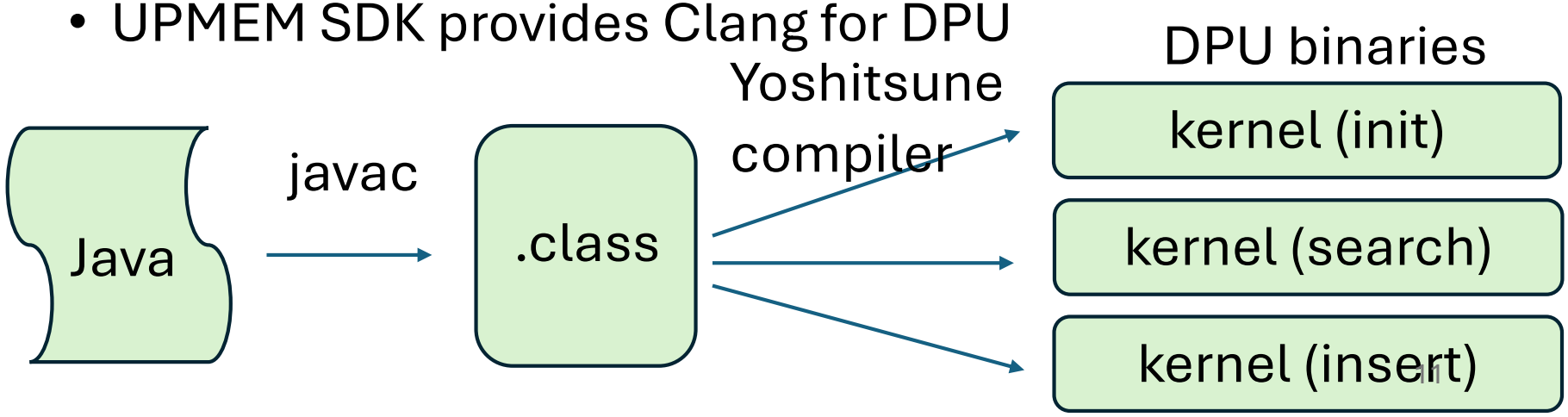
Yoshitsune's Approach

- Each DPU has separate heap in its DRAM
- Execute the same kernel method in all DPUs
- Host API guarantees that program in DPU use only objects in its own heap
  - no inter-DPU or DPU-to-CPU references



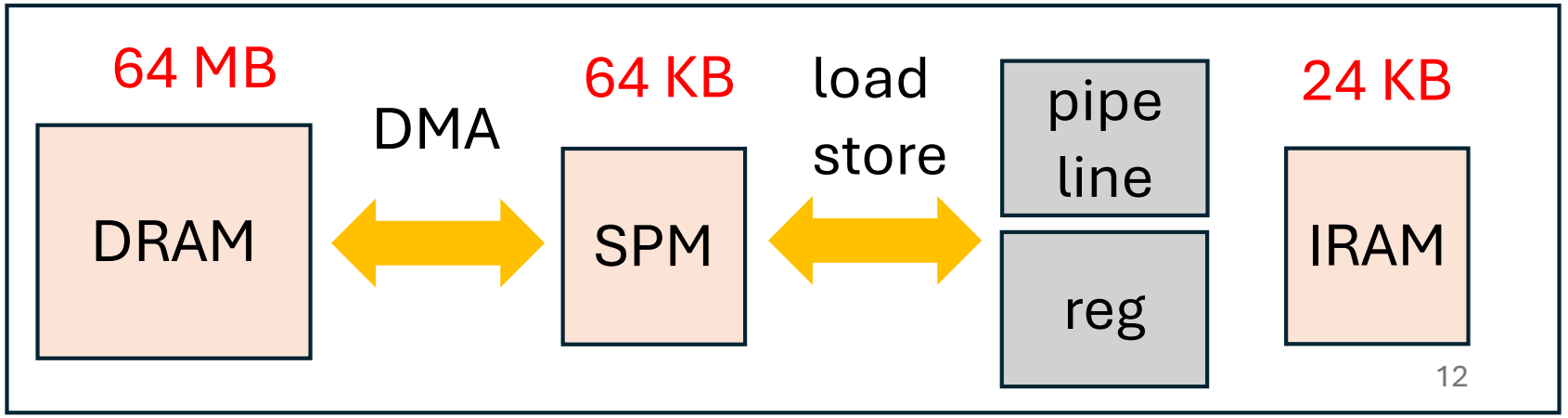
Java to DPU Compiler

- Compiles kernel methods in advance
- Compiles each kernel method to separate DPU binary
  - kernel method
  - methods that may be called from it
- Straightforward bytecode to C compiler
  - method calls are special
  - UPMEM SDK provides Clang for DPU



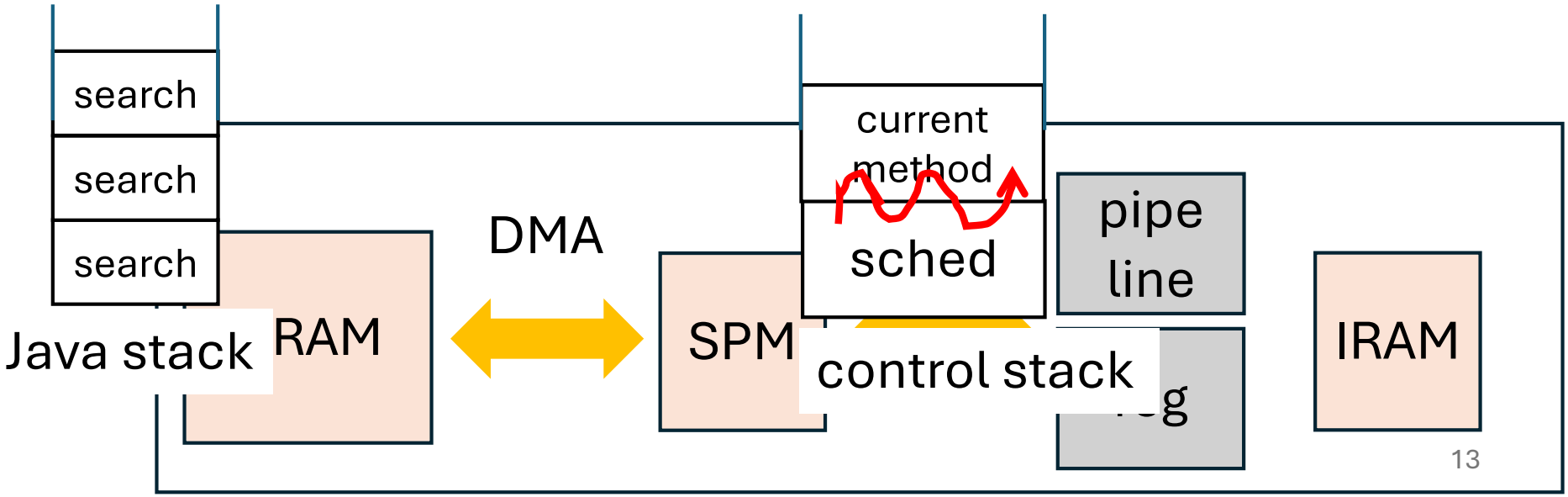
Limitations on Memory Size

- 24 KB of program memory
  - contains up to 4 K instructions
- 64 KB of SPM
  - working area + control stacks for up to 24 threads
  - need to keep control stacks small to maximize the working area



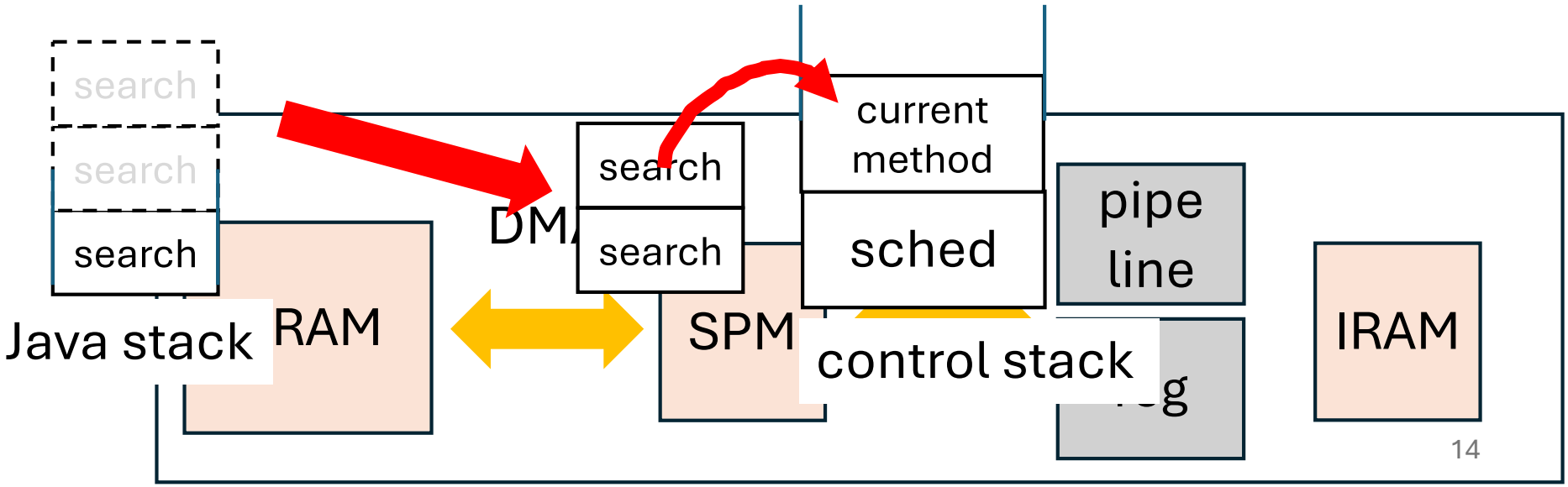
Keep Control Stack Small

- Explicit Java Stack in DRAM
  - local variable, operand stack
- Trampoline technique to limit control stack size
  - returns to scheduler to call method



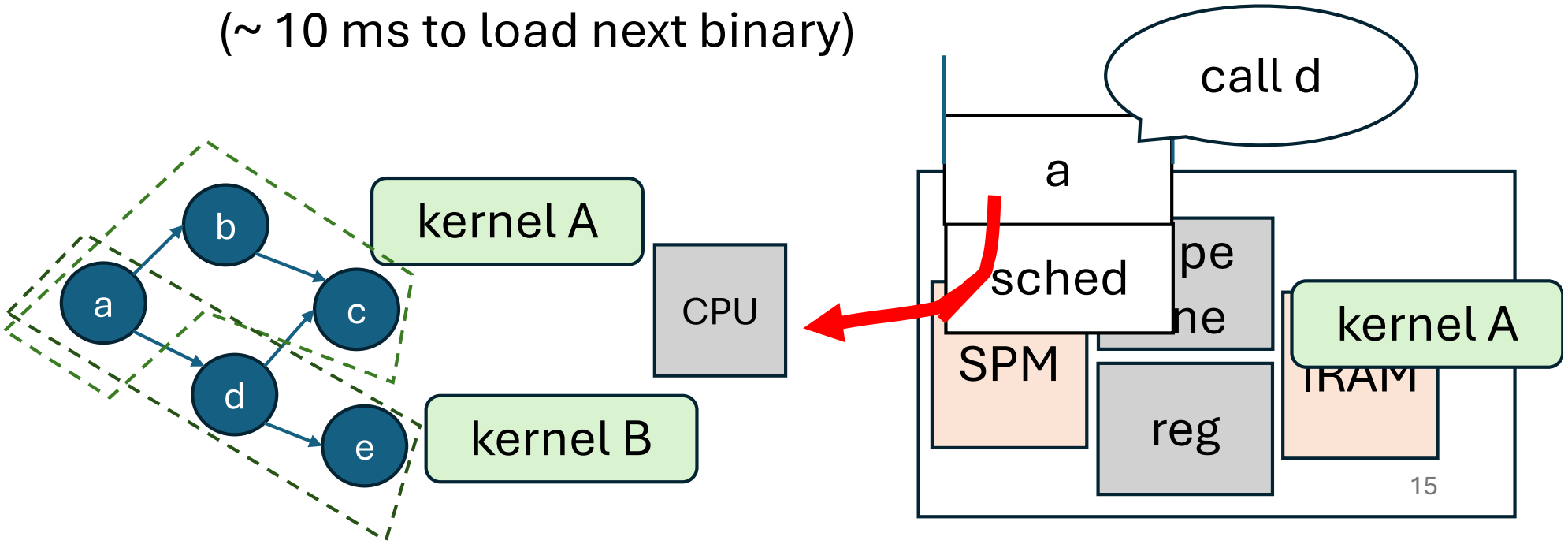
Optimization

- Move few frames to the fixed size are in SPM
  - avoid DMA overhead
  - reduce code size (DAM needs 5 more instructions)
- Cache local variables and operand stack entries in C's local variables to encourage reg. allocation



Keep DPU Binary Small

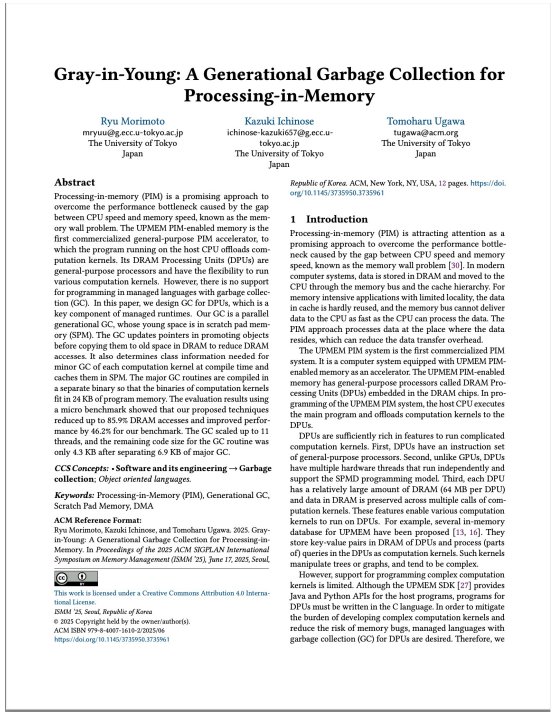
- Create DPU binary for each kernel method
- Split binary into pieces (last resort)
  - Split call graph into pieces (currently, split manually)
  - Compile each piece to a DPU binary
  - Scheduler returns to CPU to ask to load next binary (~ 10 ms to load next binary)



Runtime System in DPU

Statically linked to DPU binary

- Garbage collection
  - Mark-sweep GC
- SPMD support
  - predefined number of thread execute the same method
  - get thread ID
  - barrier synchronization



Gray-in-Young: A Generational Garbage Collection for Processing-in-Memory

Ryu Morimoto
mryuu@g.ecc.u-tokyo.ac.jp
The University of Tokyo
Japan

Kazuki Ichinose
ichinose-kazuki657@g.ecc.u-tokyo.ac.jp
The University of Tokyo
Japan

Tomoharu Ugawa
tugawa@acm.org
The University of Tokyo
Japan

Abstract

Processing-in-memory (PIM) is a promising approach to overcome the performance bottleneck caused by the gap between CPU speed and memory speed, known as the memory wall problem. The UPMEM PIM-enabled memory is the first commercialized general-purpose PIM accelerator, to which the program running on the host CPU offloads computation kernels. Its DRAM Processing Units (DPUs) are general-purpose processors and have the flexibility to run various computation kernels. However, there is no support for programming in managed languages with garbage collection (GC). In this paper, we design GC for DPUs, which is a key component of managed runtimes. Our GC is a parallel generational GC, whose young space is in scratch pad memory (SPM). The GC updates pointers in promoting objects before copying them to old space in DRAM to reduce DRAM accesses. It also determines class information needed for minor GC of each computation kernel at compile time and caches them in SPM. The major GC routines are compiled in a separate binary so that the binaries of computation kernels fit in 24 KB of program memory. The evaluation results using a micro benchmark showed that our proposed techniques reduced up to 85.9% DRAM accesses and improved performance by 46.2% for our benchmark. The GC scaled up to 11 threads, and the remaining code size for the GC routine was only 4.3 KB after separating 6.9 KB of major GC.

CCS Concepts: • **Software and its engineering → Garbage collection**; *Object oriented languages.*

Keywords: Processing-in-Memory (PIM), Generational GC, Scratch Pad Memory, DMA

ACM Reference Format:
Ryu Morimoto, Kazuki Ichinose, and Tomoharu Ugawa. 2025. Gray-in-Young: A Generational Garbage Collection for Processing-in-Memory. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on Memory Management (ISMM '25), June 17, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3735950.3735961

ISMM '25, Seoul, Republic of Korea

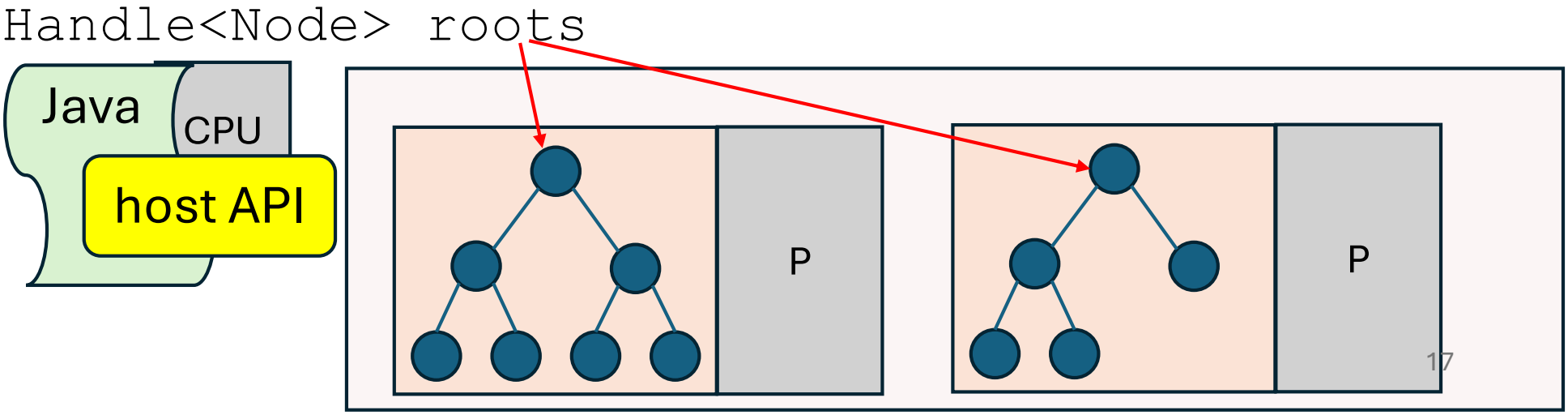
ACM ISBN 979-8-4007-1610-2/2025/06
https://doi.org/10.1145/3735950.3735961

Generational GC for DPU
to appear in ISMM 2025

Host API (not implemented yet)

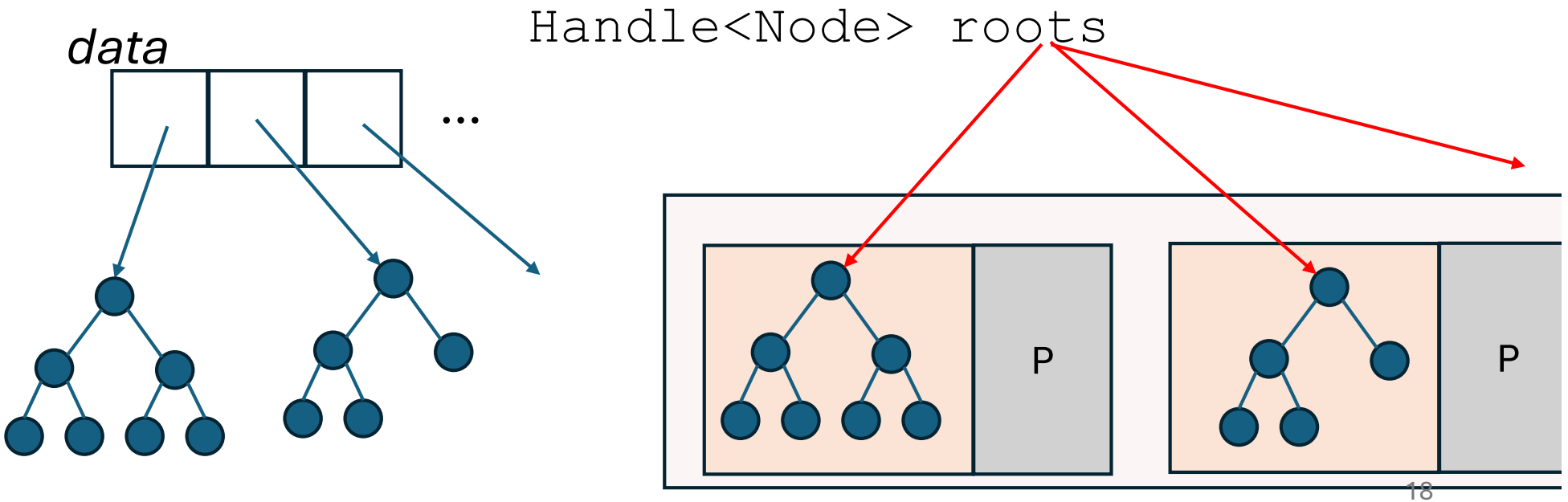
Guarantees kernel method use only objects in its own heap

- Aggregated handle
  - Handle to objects in all DPUs of the same class
- Arguments and return values of kernel method must be aggregated handles



Explicit Data Transfer

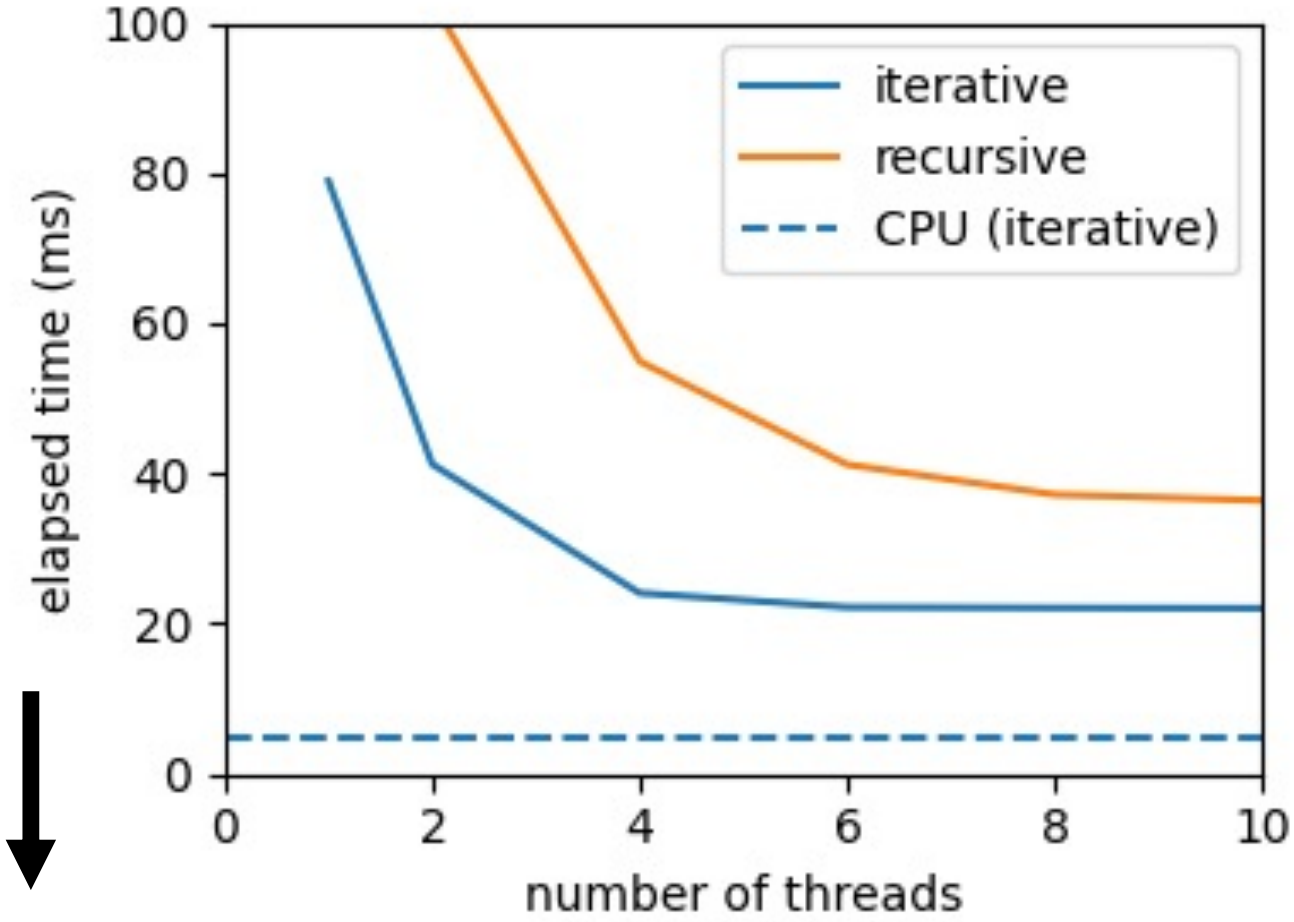
- `Handle<T> send(T[] data);`
  - Makes copy of each element of *data* to each DPU
  - Returns the aggregated handle to those copies



Preliminary Evaluation

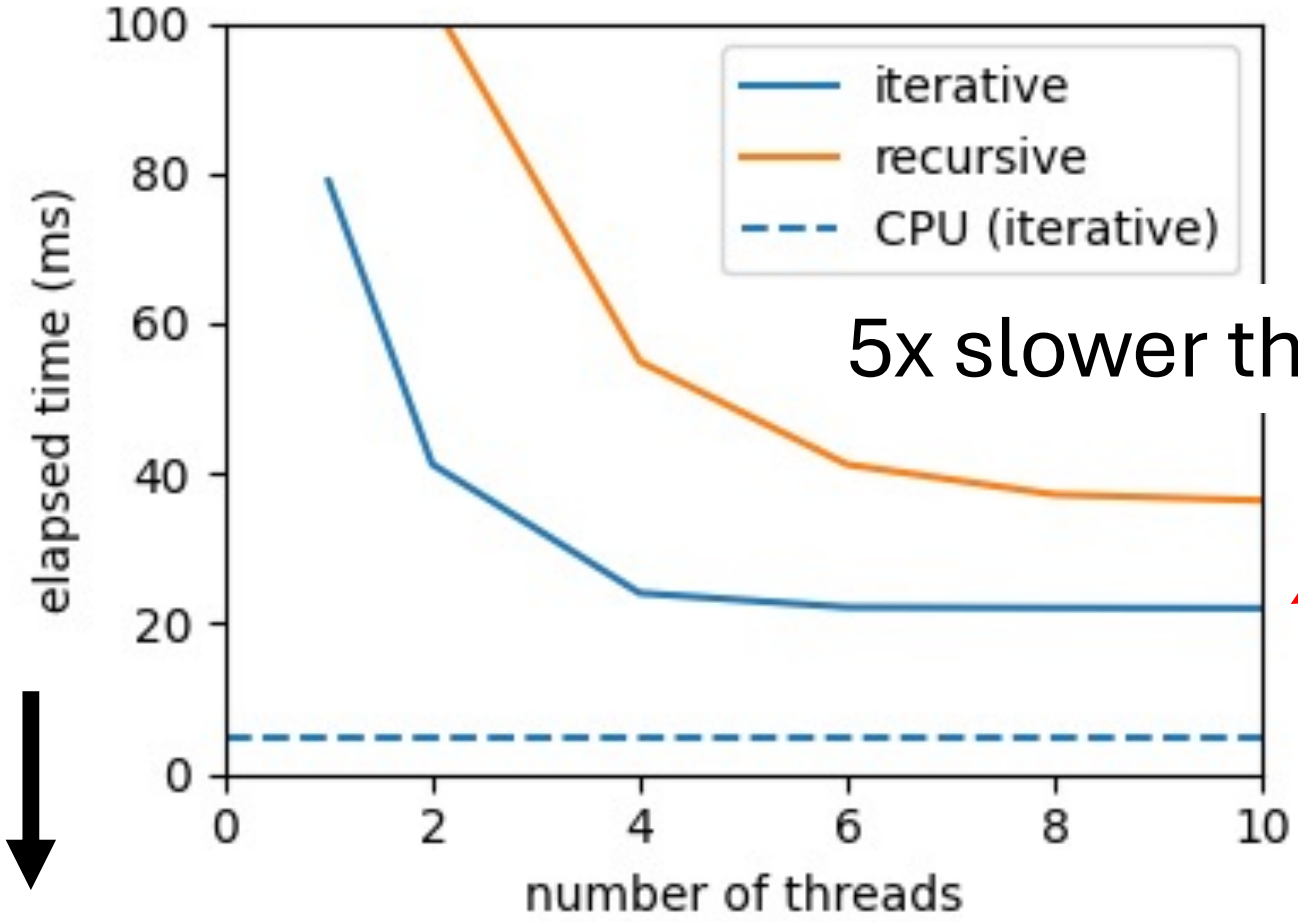
- Evaluated performance in single DPU
  - Compared to execution of 2500 x larger problem in normal JVM on Xeon 6354 x 2
    (3 GHz, 72 threads, 16 memory channels)
- Binary tree search
  - single tree with 30 K key-value pairs for DPU
  - 2500 trees of the same size for normal JVM
  - 10 K search queries
  - 100 iterations

Results



lower is better

Results



5x slower than normal JVM

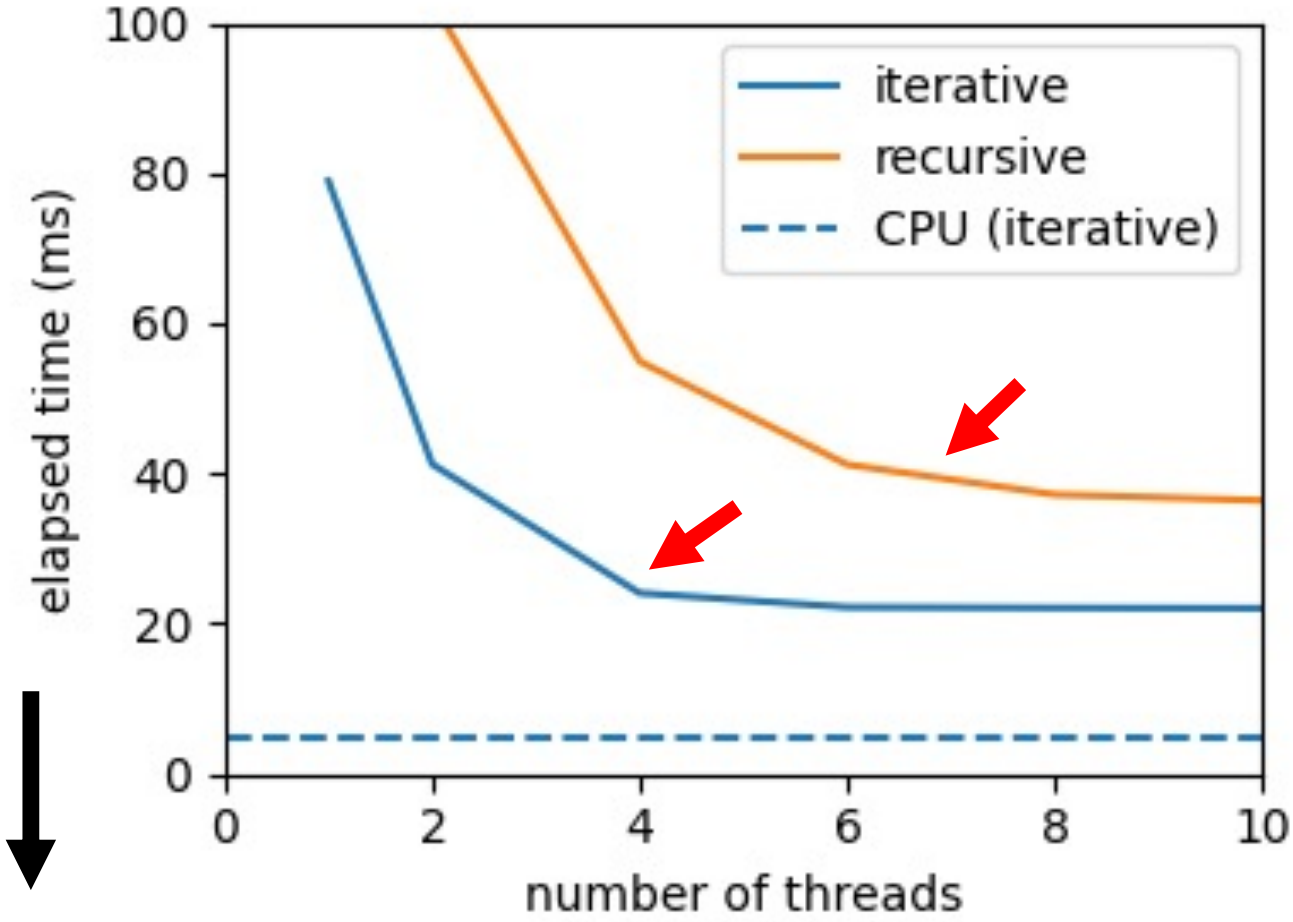
22.1

4.8

lower is better

Recursive method was slower

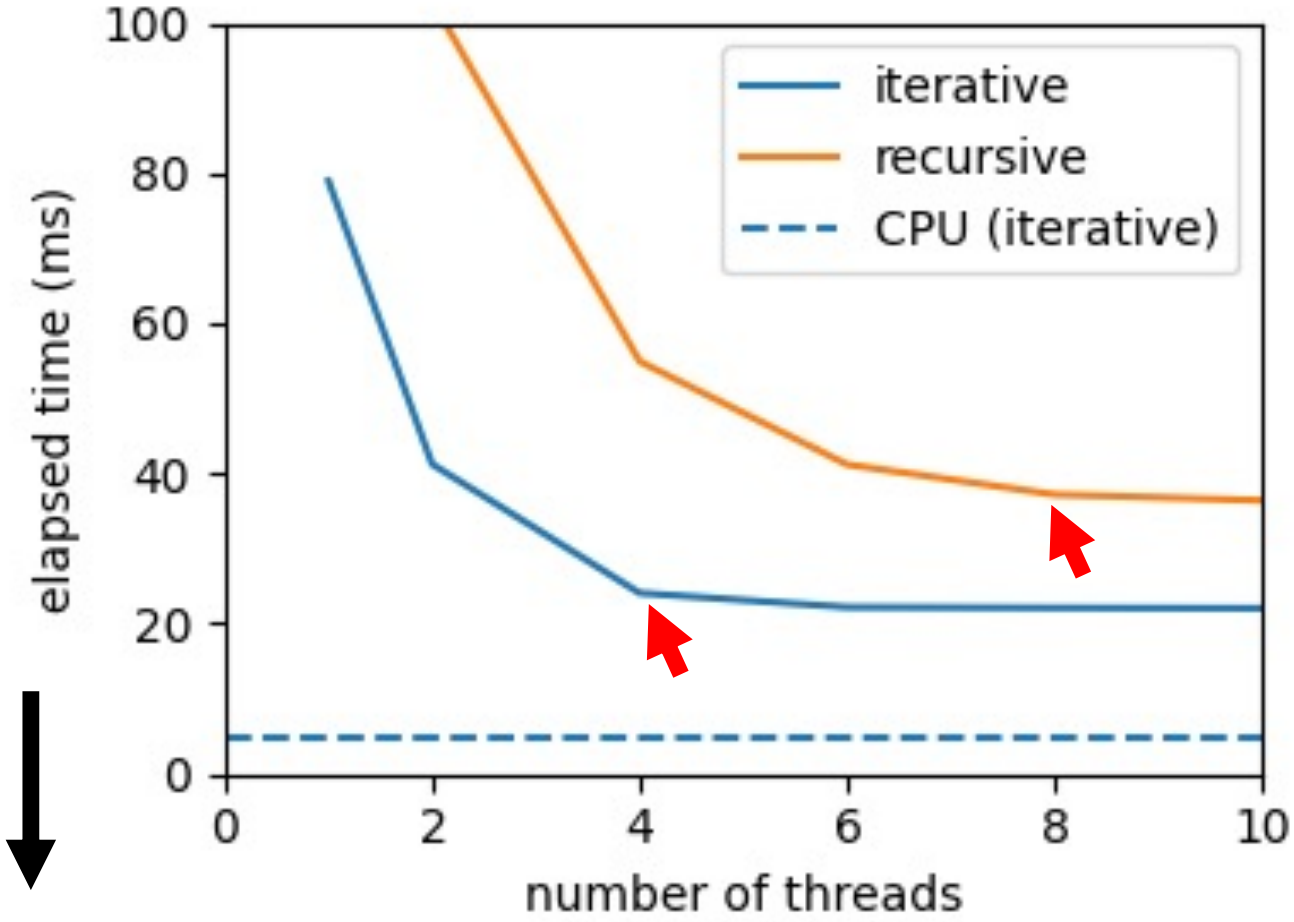
- trampoline
- more DRAM access for method dispatch



lower is better

Did not scale well

DMA controller was bottleneck



lower is better

Related Work

- Offload regular computations to accelerators
  - Tornado VM [Clarkson 2018]
  - Heterogeneous Accelerator Toolkit (HAT)

- Data is stored in the format optimized for the accelerators
- Data types that are efficiently handled are limited

Conclusion

- Developing Java framework to offload irregular computation to PIM accelerators
- Prototype was slow
- Feature work
  - Simpler code for smaller kernel
    - no trampoline
    - no preparation for switching binaries
  - Reduce DRAM accesses
    - cache objects in SPM
  - Implementation of host API
  - Design of higher level host API (like stream API)